

The Program in Action

Even though it's a module, *sevseg.py* outputs a sample demo of the digits it produces when you run the program directly. The output will look like this:

This module is meant to be imported rather than run.

For example, this code:

```
import sevseg
myNumber = sevseg.getSevSegStr(42, 3)
print(myNumber)
```

Will print 42, zero-padded to three digits:

```
  _  _  _
|_  |_  |_
|_  |_  |_
```

How It Works

The `getSevSegStr()` function first creates a list of three strings. The strings represent the top, middle, and bottom row of the numeric digits. Lines 27 to 75 have a long list of `if-elif` statements for each digit (and the decimal point and minus sign) that concatenates the rows of each digit to these strings. These three strings are joined together with newlines on line 84 so that the function returns a single multiline string suitable to pass to `print()`.

```
1. """Sevseg, by Al Sweigart al@inventwithpython.com
2. A seven-segment number display module, used by the Countdown and Digital
3. Clock programs.
4. More info at https://en.wikipedia.org/wiki/Seven-segment_display
5. View this code at https://nostarch.com/big-book-small-python-projects
6. Tags: short, module"""
7.
8. """A labeled seven-segment display, with each segment labeled A to G:
9.   A
10.  |__|  Each digit in a seven-segment display:
11. F      B
12. |__G__|  |__| |__| |__| |__| |__| |__| |__| |__|
13. |      |  |__| |__| |__| |__| |__| |__| |__|
14. E      C
15. |__D__|"""
16.
17.
18. def getSevSegStr(number, minWidth=0):
19.     """Return a seven-segment display string of number. The returned
20.     string will be padded with zeros if it is smaller than minWidth."""
21.
22.     # Convert number to string in case it's an int or float:
23.     number = str(number).zfill(minWidth)
24.
25.     rows = ['', '', '']
26.     for i, numeral in enumerate(number):
27.         if numeral == '.': # Render the decimal point.
28.             rows[0] += ' '
29.         else:
30.             digit = '0123456789.'.index(numeral)
31.             rows[0] += _SEVSEG_TOP[digit]
32.             rows[1] += _SEVSEG_MIDDLE[digit]
33.             rows[2] += _SEVSEG_BOTTOM[digit]
```